Bilgisayar Programlama

Diziler (Arrays)

İçindekiler

1. Diziler Kavramı ve Temelleri
   - Dizi Nedir, Neden Gereklidir?
   - Bellek Yerleşimi, Tanımlama ve Başlatma
2. Tek Boyutlu Diziler ve Veri İşleme
   - Elemanlara Erişim, Döngülerle Manipülasyon, Sınır Aşımı
3. Çok Boyutlu Diziler ve Matris İşlemleri
   - 2D Diziler, Row-major Order, Matris Çarpımı
4. Karakter Dizileri ve Metin İşleme (Strings)
   - Null-terminator, String Okuma/Yazma, Karakter İşleme
5. string.h Kütüphanesi ve Fonksiyonları
   - strlen, strcpy, strcat, strcmp, strncpy, strncat, strstr, memcpy, memset
6. Uygulamalı Mekatronik Senaryoları
   - Hareketli Ortalama, Matris Çarpımı, Arama ve Sıralama

BÖLÜM 1

Diziler Kavramı ve Temelleri

1.1. Dizi (Array) Nedir?

Dizi, **aynı veri tipindeki** birden fazla değişkenin **tek bir isim altında**, bellekte **ardışık (contiguous)** olarak tutulmasıdır.

Analoji: Bir apartman düşünün. Her daire aynı yapıdadır (aynı veri tipi), her birinin kapı numarası vardır (indeks) ve daireler yan yana sıralıdır (ardışık bellek).

Kavram	Apartman Benzetmesi	Dizi Karşılığı
Yapı	Apartman binası	Dizi değişkeni
Birim	Her bir daire	Dizi elemanı
Numara	Kapı numarası (1, 2, 3...)	İndeks (0, 1, 2...)
Kural	Tüm daireler aynı tip	Tüm elemanlar aynı veri tipi

1.2. Mekatronikteki Yeri: Neden Dizilere İhtiyaç Duyarız?

Mekatronik mühendisliğinde diziler **vazgeçilmezdir**:

Senaryo	Açıklama
Sensör Verileri	Bir sıcaklık sensöründen gelen son 100 okumayı tutma
Motor Kontrolü	PWM değerlerini dizide saklama ve rampa oluşturma
Görüntü İşleme	Kamera piksellerini 2B dizi olarak işleme
Robotik	Kinematik dönüşüm matrislerini saklama (4×4 matris)
Haberleşme	UART/SPI üzerinden gelen veri paketlerini tamponlama

Tek değişkenlerle bu mümkün değildir. 100 sensör okuması için 100 ayrı değişken tanımlamak → **Pratik değil!**

1.3. Tek Değişken vs Dizi Karşılaştırması

❌ Dizisiz yaklaşım — 5 sensör okumak için:

```
int sensor1 = 23, sensor2 = 25, sensor3 = 22;
int sensor4 = 24, sensor5 = 26;
int ort = (sensor1 + sensor2 + sensor3 + sensor4 + sensor5) / 5;
// 100 sensör olsa? 100 değişken mi tanımlayacağız?
```

✅ Diziyle yaklaşım — n tane sensör okumak için:

```
int sensor[5] = {23, 25, 22, 24, 26};
int toplam = 0;
for (int i = 0; i < 5; i++) {
    toplam += sensor[i];
}
int ort = toplam / 5;
// 100 sensör olsa bile sadece boyutu değiştirmek yeterli!
```

1.4. Bellek Yerleşimi (Memory Layout)

Dizi elemanları bellekte **ardışık bloklar** halinde yerleşir. Her eleman, veri tipinin boyutu kadar yer kaplar:

```
Bellek Adresi:   1000    1004    1008    1012    1016

  int dizi[5]:  | 23    | 25    | 22    | 24    | 26    |

  İndeks:        [0]     [1]     [2]     [3]     [4]
```

Adres Hesaplama Formülü:

```
eleman_adresi = baslangic_adresi + (indeks × sizeof(veri_tipi))
```

Örnek: `dizi[3]` → `1000 + (3 × 4) = 1012`

1.5. Farklı Veri Tiplerinde Bellek Kullanımı

Veri Tipi	Eleman Boyutu	10 Elemanlık Dizi	Toplam Bellek
`char`	1 byte	`char str[10]`	10 byte
`int`	4 byte	`int dizi[10]`	40 byte
`float`	4 byte	`float volt[10]`	40 byte
`double`	8 byte	`double hassas[10]`	80 byte

sizeof operatörü ile doğrulama:

```
1  int dizi[10];
2  printf("Eleman boyutu: %lu byte\n", sizeof(dizi[0]));  // 4
3  printf("Dizi boyutu  : %lu byte\n", sizeof(dizi));      // 40
4  printf("Eleman sayisi: %lu\n", sizeof(dizi) / sizeof(dizi[0])); // 10
```

1.6. Dizi Tanımlama (Declaration)

Dizi tanımlamak için **veri tipi**, **dizi adı** ve **eleman sayısı (boyut)** belirtilir:

```
veri_tipi dizi_adi[eleman_sayisi];
```

Örnekler:

```
int notlar[50];          // 50 elemanlık int dizi
float sicaklik[100];     // 100 elemanlık float dizi
double voltaj[1024];     // 1024 elemanlık double dizi
char isim[30];           // 30 karakterlik karakter dizisi
```

⚠️ Kurallar:

- Boyut **pozitif tam sayı** olmalıdır (C99'da değişken boyut mümkün, ancak gömülü sistemlerde önerilmez)
- Dizi adı geçerli bir C tanımlayıcısı olmalıdır
- Boyut, derleme zamanında bilinmelidir (standart C'de)

1.7. Dizi Başlatma (Initialization)

Diziler tanımlama anında değer atanarak başlatılabilir:

```
// 1. Tam başlatma — tüm elemanlar belirtilir
int a[5] = {10, 20, 30, 40, 50};

// 2. Kısmi başlatma — geri kalan elemanlar 0 olur
int b[5] = {10, 20};  // b = {10, 20, 0, 0, 0}

// 3. Sıfır başlatma — tüm elemanlar 0
int c[5] = {0};        // c = {0, 0, 0, 0, 0}

// 4. Boyutsuz başlatma — derleyici boyutu saptar
int d[] = {5, 10, 15, 20};  // Boyut otomatik: 4

// 5. Designated initializer (C99)
int e[5] = {[2] = 30, [4] = 50};  // e = {0, 0, 30, 0, 50}
```

1.8. Başlatılmamış Dizi Tehlikesi

⚠️ **Kritik Uyarı:** Başlatılmamış yerel (local) diziler **çöp değer (garbage value)** içerir!

```c
int main() {
    int dizi[5];  // Başlatılmadı!
    for (int i = 0; i < 5; i++) {
        printf("dizi[%d] = %d\n", i, dizi[i]);
    }
    return 0;
}
```

```
Olası Çıktı (her çalıştırmada farklı!):
dizi[0] = 32764
dizi[1] = -1084227584
dizi[2] = 0
dizi[3] = 4196048
dizi[4] = 0
```

Çözüm: Diziyi her zaman `{0}` ile başlatın veya döngüyle değer atayın.

1.9. Sabit Boyutlu Dizi Tanımlama (#define ve const)

Dizi boyutunu sihirli sayı olarak yazmak yerine sabit kullanmak iyi pratiktir:

```
// ✅ İyi pratik: #define ile sabit boyut
#define MAX_SENSOR 100

int sensorVerileri[MAX_SENSOR];
for (int i = 0; i < MAX_SENSOR; i++) {
    sensorVerileri[i] = 0;
}

// ✅ Alternatif: const (ancak bazı derleyicilerde dizi boyutu için sınırlı destek)
// const int BOYUT = 100;
// int dizi[BOYUT];  // C99 VLA — gömülü sistemlerde dikkatli kullanın
```

Neden? Boyut değiştiğinde sadece `#define` satırını güncellemek yeterlidir. Kodun 50 farklı yerinde "100" aramak gerekmez.

BÖLÜM 2

Tek Boyutlu Diziler ve Veri İşleme

2.1. Elemanlara Erişim ve İndeksleme

C dilinde dizi indeksleri 0'dan başlar:

```
Eleman:    23    25    22    24    26
İndeks:   [0]   [1]   [2]   [3]   [4]    ← Geçerli indeksler
                                  [5]    ← GEÇERSİZ! (Sınır dışı)
```

```c
int sensor[5] = {23, 25, 22, 24, 26};

// Okuma
printf("İlk eleman : %d\n", sensor[0]);    // 23
printf("Son eleman : %d\n", sensor[4]);    // 26
// sensor[n-1] her zaman son elemandır

// Yazma (değer atama)
sensor[2] = 99;  // 3. eleman artık 99
```

⚠️ **Sık yapılan hata:** Son eleman `sensor[5]` değil, `sensor[4]` 'tür!

2.2. Döngülerle Dizi Okuma ve Yazma

Diziler ve `for` döngüsü doğal ortaklardır:

```c
#define N 5
int sensor[N];

// 1. Diziye değer yazma (kullanıcıdan okuma)
printf("5 adet sensör değeri giriniz:\n");
for (int i = 0; i < N; i++) {
    printf("sensor[%d]: ", i);
    scanf("%d", &sensor[i]);
}

// 2. Diziyi okuma (ekrana yazdırma)
printf("Girilen değerler: ");
for (int i = 0; i < N; i++) {
    printf("%d ", sensor[i]);
}
printf("\n");
```

Dikkat: `scanf` ile diziye değer okurken `&sensor[i]` (adres operatörü) kullanılır.

2.3. Dizi Üzerinde Temel İşlemler

```c
#define N 10
int dizi[N] = {45, 72, 18, 93, 56, 31, 87, 64, 29, 41};

// TOPLAM ve ORTALAMA
int toplam = 0;
for (int i = 0; i < N; i++) {
    toplam += dizi[i];
}
double ort = (double)toplam / N;
printf("Toplam: %d, Ortalama: %.2f\n", toplam, ort);

// MİNİMUM ve MAKSİMUM
int min = dizi[0], max = dizi[0];
int minIdx = 0, maxIdx = 0;
for (int i = 1; i < N; i++) {
    if (dizi[i] < min) { min = dizi[i]; minIdx = i; }
    if (dizi[i] > max) { max = dizi[i]; maxIdx = i; }
}
printf("Min: %d (indeks %d)\n", min, minIdx);
printf("Max: %d (indeks %d)\n", max, maxIdx);
```

2.4. İzleme Tablosu: Min/Max Bulma Algoritması

`dizi = {45, 72, 18, 93, 56}` için adım adım izleme:

Adım	`i`	`dizi[i]`	`min`	`max`	Açıklama
Baş.	—	—	45	45	İlk eleman referans
1	1	72	45	**72**	Yeni max bulundu
2	2	18	**18**	72	Yeni min bulundu
3	3	93	18	**93**	Yeni max bulundu
4	4	56	18	93	Değişiklik yok

Sonuç: Min = 18 (indeks 2), Max = 93 (indeks 3)

2.5. Doğrusal Arama (Linear Search)

Bir dizide belirli bir değeri aramak için en basit yöntem:

```c
#define N 8
int dizi[N] = {14, 28, 7, 42, 35, 21, 56, 63};
int aranan = 35;
int bulundu = -1;  // -1 = bulunamadı

for (int i = 0; i < N; i++) {
    if (dizi[i] == aranan) {
        bulundu = i;   // Bulunan indeksi kaydet
        break;         // İlk bulduğunda dur
    }
}

if (bulundu != -1)
    printf("%d değeri indeks %d'de bulundu.\n", aranan, bulundu);
else
    printf("%d değeri dizide bulunamadı.\n", aranan);
```

Karmaşıklık: O(n) — En kötü durumda tüm diziyi tarar.

2.6. Sınır Aşımı (Out of Bounds) Tehlikesi

C dilinde dizi sınırları **derleyici tarafından kontrol EDİLMEZ**. Bu, ciddi güvenlik açıklarına yol açar:

```c
int dizi[5] = {10, 20, 30, 40, 50};

// ❌ TEHLİKELİ: Sınır dışı erişim (Buffer Overflow)
dizi[5] = 999;    // Tanımsız davranış! Bellekte başka verinin üzerine yazar
dizi[-1] = 888;   // Negatif indeks! Diziden önceki bellek alanına yazar
dizi[100] = 777;  // Tamamen rastgele bir bellek alanına yazıyor!

// Program çökebilir, yanlış sonuç verebilir veya güvenlik açığı oluşur
```

⚠️ **Buffer overflow** saldırıları, tarihte en çok kullanılan siber güvenlik açıklarındandır (Morris Worm, 1988).

2.7. Güvenli Dizi Kullanımı İçin Kurallar

Kural	Açıklama
✅ `#define` ile boyut	Boyutu tek bir yerde tanımlayın
✅ `i < N` koşulu	Döngülerde sınırı aşmayın
✅ Girdi doğrulama	Kullanıcı indeksi kontrol edin
❌ `i <= N` kullanmayın	Off-by-one hatası: son indeks `N-1`'dir
❌ Negatif indeks	Tanımsız davranışa yol açar

```c
// ✅ Güvenli erişim örneği
int indeks;
printf("Hangi indeksi görmek istersiniz? (0-%d): ", N-1);
scanf("%d", &indeks);

if (indeks >= 0 && indeks < N)
    printf("dizi[%d] = %d\n", indeks, dizi[indeks]);
else
    printf("HATA: Geçersiz indeks!\n");
```

2.8. Diziyi Ters Çevirme (Reverse)

Swap (takas) yöntemiyle diziyi yerinde ters çevirme:

```c
#define N 6
int dizi[N] = {10, 20, 30, 40, 50, 60};

// Ters çevirme: Baş ve son elemanları takas et
for (int i = 0; i < N / 2; i++) {
    int gecici = dizi[i];
    dizi[i] = dizi[N - 1 - i];
    dizi[N - 1 - i] = gecici;
}
// Sonuç: {60, 50, 40, 30, 20, 10}
```

```
Adım 1: [10, 20, 30, 40, 50, 60]  → dizi[0] ↔ dizi[5]
Adım 2: [60, 20, 30, 40, 50, 10]  → dizi[1] ↔ dizi[4]
Adım 3: [60, 50, 30, 40, 20, 10]  → dizi[2] ↔ dizi[3]
Sonuç:  [60, 50, 40, 30, 20, 10]  ✅
```

2.9. Dizi Kopyalama

C dilinde diziler `=` operatörü ile **doğrudan kopyalanamaz**. Eleman eleman kopyalanmalıdır:

```c
#define N 5
int kaynak[N] = {11, 22, 33, 44, 55};
int hedef[N];

// ❌ HATALI: Diziler doğrudan atanamaz
// hedef = kaynak;  // Derleme hatası!

// ✅ DOĞRU: Döngü ile eleman eleman kopyalama
for (int i = 0; i < N; i++) {
    hedef[i] = kaynak[i];
}

// ✅ Alternatif: memcpy kullanımı (string.h gerekir)
// #include <string.h>
// memcpy(hedef, kaynak, sizeof(kaynak));
```

2.10. Dizide Eleman Kaydırma (Shift)

Bir elemanı silmek veya eklemek için kaydırma işlemi gerekir:

```
#define MAX 10
int dizi[MAX] = {10, 20, 30, 40, 50};
int n = 5;  // Mevcut eleman sayısı

// Sola kaydırma: İndeks 2'deki elemanı sil
int silinecek_idx = 2;
for (int i = silinecek_idx; i < n - 1; i++) {
    dizi[i] = dizi[i + 1];  // Sağdaki elemanı sola kopyala
}
n--;  // Eleman sayısını azalt
// Sonuç: {10, 20, 40, 50} — n=4

// Sağa kaydırma: İndeks 1'e yeni eleman ekle
int eklenecek_idx = 1, yeni_deger = 99;
for (int i = n; i > eklenecek_idx; i--) {
    dizi[i] = dizi[i - 1];  // Soldaki elemanı sağa kopyala
}
dizi[eklenecek_idx] = yeni_deger;
n++;
// Sonuç: {10, 99, 20, 40, 50} — n=5
```

BÖLÜM 3

Çok Boyutlu Diziler ve Matris İşlemleri

3.1. İki Boyutlu Diziler (2D Arrays)

2D dizi, satır ve sütunlardan oluşan bir tablo (matris) yapısıdır:

```c
// Tanımlama: veri_tipi ad[satir_sayisi][sutun_sayisi];
int matris[3][4];  // 3 satır, 4 sütun = 12 eleman

// Başlatma
int m[2][3] = {
    {1, 2, 3},    // 0. satır
    {4, 5, 6}     // 1. satır
};

// Erişim: matris[satir][sutun]
printf("%d\n", m[1][2]);  // 6 (1. satır, 2. sütun)
```

```
          Sütun 0  Sütun 1  Sütun 2
Satır 0:    1        2        3
Satır 1:    4        5        6
```

3.2. 2D Dizi Başlatma Yöntemleri

```
// 1. Tam başlatma (satır satır)
int a[2][3] = {{1, 2, 3}, {4, 5, 6}};

// 2. Düz liste (satır sırasıyla doldurulur)
int b[2][3] = {1, 2, 3, 4, 5, 6};  // Aynı sonuç

// 3. Kısmi başlatma (geri kalanlar 0)
int c[2][3] = {{1}, {4}};
// c = {{1, 0, 0}, {4, 0, 0}}

// 4. Tümü sıfır
int d[2][3] = {0};  // Tüm elemanlar 0

// 5. İlk boyut otomatik
int e[][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
// Satır sayısı = 3 (derleyici saptar)
// NOT: İkinci boyut HER ZAMAN belirtilmelidir!
```

3.3. Bellekte 2D Dizi: Row-Major Order

C dilinde 2D diziler bellekte satır öncelikli (row-major) olarak saklanır:

```
int m[2][3] = {{1,2,3}, {4,5,6}};

Mantıksal:                Fiziksel (bellekte):

 ┌───┬───┬───┐  satır 0   ┌───┬───┬───┬───┬───┬───┐
 │ 1 │ 2 │ 3 │            │ 1 │ 2 │ 3 │ 4 │ 5 │ 6 │
 ├───┼───┼───┤            └───┴───┴───┴───┴───┴───┘
 │ 4 │ 5 │ 6 │  satır 1  [0][0] [0][1] [0][2] [1][0] [1][1] [1][2]
 └───┴───┴───┘              ← satır 0 →   ← satır 1 →
```

Adres hesabı: `m[i][j]` → Offset = `i × sütun_sayısı + j`

`m[1][2]` → `1 × 3 + 2 = 5` → Bellekteki 5. eleman → 6 ✅

3.4. İç İçe Döngülerle 2D Dizi İşlemleri

2D dizilerde dış döngü **satırları**, iç döngü **sütunları** tarar:

```c
#define SATIR 3
#define SUTUN 4

int matris[SATIR][SUTUN];

// Değer atama
for (int i = 0; i < SATIR; i++) {
    for (int j = 0; j < SUTUN; j++) {
        matris[i][j] = i * SUTUN + j + 1;
    }
}

// Yazdırma
printf("Matris:\n");
for (int i = 0; i < SATIR; i++) {
    for (int j = 0; j < SUTUN; j++) {
        printf("%4d", matris[i][j]);
    }
    printf("\n");
}
```

3.5. 2D Dizi Örneği: Sınıf Not Tablosu

```c
#define OGRENCI 4
#define DERS 3

int notlar[OGRENCI][DERS] = {
    {85, 90, 78},   // Öğrenci 0
    {92, 88, 95},   // Öğrenci 1
    {67, 72, 80},   // Öğrenci 2
    {55, 60, 45}    // Öğrenci 3
};

// Her öğrencinin ortalamasını hesapla
for (int i = 0; i < OGRENCI; i++) {
    int toplam = 0;
    for (int j = 0; j < DERS; j++) {
        toplam += notlar[i][j];
    }
    double ort = (double)toplam / DERS;
    printf("Öğrenci %d ortalaması: %.1f %s\n",
        i, ort, ort >= 60 ? "(Geçti)" : "(Kaldı)");
}
```

3.6. Matris Toplama

İki matrisin toplanması: `C[i][j] = A[i][j] + B[i][j]`

```
1   #define N 3
2
3   int A[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
4   int B[N][N] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
5   int C[N][N];  // Sonuç matrisi
6
7   for (int i = 0; i < N; i++) {
8       for (int j = 0; j < N; j++) {
9           C[i][j] = A[i][j] + B[i][j];
10      }
11  }
12
13  // C = {{10,10,10}, {10,10,10}, {10,10,10}}
```

Kural: Matris toplama için her iki matrisin boyutları **aynı** olmalıdır.

3.7. Matris Çarpımı (Matrix Multiplication)

İki matrisin çarpılması: `C[i][j] = Σ A[i][k] × B[k][j]`

```
#define N 3

double A[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
double B[N][N] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
double C[N][N] = {0};  // Sonuç matrisi — SIFIRLA!

for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        C[i][j] = 0;  // Güvenli sıfırlama
        for (int k = 0; k < N; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
// Karmaşıklık: O(n³) — 3 katlı iç içe döngü
```

3.8. Matris Çarpımının İzlenmesi

```
C[0][0] = A[0][0]×B[0][0] + A[0][1]×B[1][0] + A[0][2]×B[2][0]
```

```
A × B = C


┌─────────┐   ┌─────────┐   ┌──────────────┐
│ 1  2  3 │   │ 9  8  7 │   │  30  24  18  │
│ 4  5  6 │ × │ 6  5  4 │ = │  84  69  54  │
│ 7  8  9 │   │ 3  2  1 │   │ 138 114  90  │
└─────────┘   └─────────┘   └──────────────┘


C[0][0] = 1×9 + 2×6 + 3×3 = 9 + 12 + 9 = 30 ✅
C[1][2] = 4×7 + 5×4 + 6×1 = 28 + 20 + 6 = 54 ✅
```

Mekatronik Uygulaması: Robotik kolda eklem dönüşüm matrislerinin çarpılması (Denavit-Hartenberg parametreleri).

3.9. Matris Transpozu

Bir matrisin satır ve sütunlarını yer değiştirme: `T[j][i] = M[i][j]`

```c
#define SATIR 2
#define SUTUN 3

int M[SATIR][SUTUN] = {{1, 2, 3}, {4, 5, 6}};
int T[SUTUN][SATIR];  // Transpoz matrisi — boyutlar ters!

for (int i = 0; i < SATIR; i++) {
    for (int j = 0; j < SUTUN; j++) {
        T[j][i] = M[i][j];
    }
}
```

```
Orijinal (2×3):    Transpoz (3×2):

┌─────────┐        ┌───────┐
│ 1  2  3 │   →    │ 1  4  │
│ 4  5  6 │        │ 2  5  │
└─────────┘        │ 3  6  │
                   └───────┘
```

3.10. 3 Boyutlu Diziler — Kavramsal Giriş

3D diziler veri küpleri olarak düşünülebilir:

```
// 3D dizi: katman × satır × sütun
int veriKupu[3][4][5];  // 3 katman, 4 satır, 5 sütun = 60 eleman

// Örnek: RGB görüntü verisi
// unsigned char goruntu[YUKSEKLIK][GENISLIK][3]; // 3 = R, G, B kanalları
// goruntu[y][x][0] = Kırmızı değeri
// goruntu[y][x][1] = Yeşil değeri
// goruntu[y][x][2] = Mavi değeri
```

Mekatronik Uygulamaları:

- Çok katmanlı sensör veri küpleri (zaman × sensör × kanal)
- 3D uzay haritaları (x × y × z)
- Video kareleri (kare × satır × sütun)

BÖLÜM 4

Karakter Dizileri ve Metin İşleme (Strings)

4.1. C Dilinde String Kavramı

C dilinde ayrı bir `String` veri tipi **yoktur**. Bunun yerine, sonu **null-terminator** (`\0`) ile biten `char` dizileri kullanılır:

```
// String tanımlama yöntemleri
char isim1[6] = {'M', 'e', 'r', 'v', 'e', '\0'};  // Elle — \0 zorunlu!
char isim2[] = "Merve";    // Otomatik — derleyici \0 ekler
char isim3[20] = "Merve";  // Fazla alan — kalan \0 ile dolar

// Bellek görünümü:
// isim2: ['M']['e']['r']['v']['e']['\0']
// İndeks:  0    1    2    3    4    5
// Boyut: 6 byte (5 karakter + 1 null-terminator)
```

⚠️ `char s[5] = "Merve";` → HATA! 5 karakter + `\0` = 6 byte gerekir. Boyut en az 6 olmalıdır.

4.2. String Okuma ve Yazdırma

```c
char isim[50];

// Okuma yöntemleri
printf("İsminiz: ");
scanf("%s", isim);     // Boşluğa kadar okur (& gerekmez!)
// "Ali Veli" girişi → sadece "Ali" okunur

printf("İsminiz: ");
fgets(isim, 50, stdin);  // Boşluklar dahil satır sonuna kadar okur
// "Ali Veli" girişi → "Ali Veli\n" okunur

// Yazdırma
printf("Merhaba, %s!\n", isim);
puts(isim);  // Otomatik satır sonu ekler
```

Fonksiyon	Boşluk Okur	Güvenli	Satır Sonu
`scanf("%s",...)`	❌	❌	Hayır
`fgets(...)`	✅	✅	Evet (`\n` dahil)

4.3. Null-Terminator'ın Önemi

`\0` karakteri stringin nerede bittiğini belirler. Eksikse ciddi sorunlar oluşur:

```c
// ✅ DOĞRU: \0 var
char s1[] = "Merhaba";
printf("%s\n", s1);  // "Merhaba" yazdırır — \0'da durur

// ❌ HATALI: \0 unutulmuş
char s2[5] = {'H', 'e', 'l', 'l', 'o'}; // \0 YOK!
printf("%s\n", s2);  // "Hello" + bellekteki çöp veriler yazdırılır!
// Buffer overread — güvenlik açığı!
```

```
Bellek:  ['H']['e']['l']['l']['o'][???][???][???]
                                   ↑ \0 olmadığı için
                                   printf burada DURMAZ!
```

4.4. Mekatronik Uygulaması: UART Komut İşleme

Gömülü sistemlerde UART üzerinden gelen metin tabanlı komutların işlenmesi:

```c
// GPS NMEA cümlesi işleme örneği
char gps_veri[] = "$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M*47";

// AT komutu kontrolü (modem haberleşmesi)
char komut[50];
// UART'tan oku: fgets(komut, 50, uart_stream);

// Basit komut karşılaştırma
if (komut[0] == 'A' && komut[1] == 'T') {
    printf("AT komutu alindi: %s\n", komut);
    // Komutu işle...
}

// SPI veri tamponu
char spi_buffer[256] = {0};  // Tampon sıfırlanmış
```

4.5. String İşleme: Karakter Karakter Tarama

Döngü ile string üzerinde karakter karakter işlem yapma:

```c
char metin[] = "Mekatronik Muhendisligi 2026";
int harf = 0, rakam = 0, bosluk = 0, diger = 0;

for (int i = 0; metin[i] != '\0'; i++) {
    if ((metin[i] >= 'A' && metin[i] <= 'Z') ||
        (metin[i] >= 'a' && metin[i] <= 'z'))
        harf++;
    else if (metin[i] >= '0' && metin[i] <= '9')
        rakam++;
    else if (metin[i] == ' ')
        bosluk++;
    else
        diger++;
}
printf("Harf: %d, Rakam: %d, Boşluk: %d, Diğer: %d\n",
       harf, rakam, bosluk, diger);
// Harf: 23, Rakam: 4, Boşluk: 2, Diğer: 0
```

4.6. Büyük/Küçük Harf Dönüşümü

```c
#include <ctype.h>  // toupper(), tolower(), isalpha() vb.

char metin[] = "Hello World 123";

// Büyük harfe çevir
for (int i = 0; metin[i] != '\0'; i++) {
    metin[i] = toupper(metin[i]);
}
printf("%s\n", metin);  // "HELLO WORLD 123"

// Küçük harfe çevir
for (int i = 0; metin[i] != '\0'; i++) {
    metin[i] = tolower(metin[i]);
}
printf("%s\n", metin);  // "hello world 123"

// Manuel dönüşüm (ctype.h olmadan):
// Büyük → Küçük: ch = ch + ('a' - 'A') = ch + 32
// Küçük → Büyük: ch = ch - 32
```

4.7. ctype.h Karakter Test Fonksiyonları

Fonksiyon	Açıklama	Örnek
`isalpha(c)`	Harf mi? (a-z, A-Z)	`isalpha('A')` → true
`isdigit(c)`	Rakam mı? (0-9)	`isdigit('5')` → true
`isalnum(c)`	Harf veya rakam mı?	`isalnum('x')` → true
`isupper(c)`	Büyük harf mi?	`isupper('A')` → true
`islower(c)`	Küçük harf mi?	`islower('a')` → true
`isspace(c)`	Boşluk karakteri mi?	`isspace(' ')` → true
`toupper(c)`	Büyük harfe çevir	`toupper('a')` → `'A'`
`tolower(c)`	Küçük harfe çevir	`tolower('Z')` → `'z'`

⚠️ Bu fonksiyonlar `<ctype.h>` kütüphanesinde tanımlıdır, `<string.h>` ile karıştırmayın.

BÖLÜM 5

`string.h` Kütüphanesi ve Fonksiyonları

⚠️ Not: Bu bölümdeki fonksiyon prototiplerinde `char *`, `const char *` gibi ifadeler göreceksiniz. Bunlar pointer (işaretçi) notasyonudur ve ileriki haftalarda detaylı işlenecektir. Şimdilik sadece "dizinin bellek adresini alan parametre" olarak düşünmeniz yeterlidir.

5.1. string.h Nedir?

C dilinde string (karakter dizisi) işlemleri için standart kütüphanedir. Diziler üzerinde **kopyalama**, **birleştirme**, **karşılaştırma**, arama ve **bellek işlemleri** yapan fonksiyonları içerir.

```
#include <string.h>  // Tüm string fonksiyonlarını kullanmak için
```

Temel Fonksiyon Kategorileri:

Kategori	Fonksiyonlar
Uzunluk	`strlen`
Kopyalama	`strcpy`, `strncpy`, `memcpy`, `memmove`
Birleştirme	`strcat`, `strncat`
Karşılaştırma	`strcmp`, `strncmp`, `memcmp`
Arama	`strchr`, `strrchr`, `strstr`
Bellek İşlemleri	`memset`, `memcpy`, `memmove`

5.2. strlen — String Uzunluğu

Fonksiyon Tanımı:

```c
size_t strlen(const char *str);
// str   : Uzunluğu hesaplanacak string
// Döndürür: \0 hariç karakter sayısı (size_t = unsigned int)
```

Kullanım Örneği:

```c
#include <string.h>

char isim[] = "Mekatronik";
int uzunluk = strlen(isim);  // 10

printf("Uzunluk: %d\n", uzunluk);        // 10
printf("sizeof : %lu\n", sizeof(isim));   // 11 (\0 dahil!)
```

⚠️ `strlen` ≠ `sizeof` → `strlen` sadece karakterleri sayar, `sizeof` bellekteki toplam boyutu (`\0` dahil) verir.

5.3. strlen — Nasıl Çalışır?

`strlen` fonksiyonu, `\0` (null-terminator) bulana kadar karakter karakter ilerler:

```c
// strlen'in iç çalışma mantığı (basitleştirilmiş)
int my_strlen(const char *str) {
    int sayac = 0;
    while (str[sayac] != '\0') {
        sayac++;
    }
    return sayac;
}
```

```
str:  ['M']['e']['k']['a']['t']['r']['o']['n']['i']['k']['\0']
        ↑                                                ↑
      sayac=0                                        sayac=10 STOP!

Sonuç: 10
```

5.4. strcpy — String Kopyalama

Fonksiyon Tanımı:

```
char *strcpy(char *hedef, const char *kaynak);
// hedef   : Kopyalanacak hedef dizi (yeterli boyutta olmalı!)
// kaynak  : Kopyalanacak kaynak string
// Döndürür: hedef dizisinin adresini
```

Kullanım Örneği:

```
char kaynak[] = "Merhaba";
char hedef[20];     // Yeterli boyut ayrılmalı!

strcpy(hedef, kaynak);
printf("%s\n", hedef);  // "Merhaba"
```

⚠️ TEHLİKE: `strcpy` hedef dizinin boyutunu **kontrol etmez**! Kaynak, hedeften büyükse **buffer overflow** oluşur.

```
char kisa[5];
strcpy(kisa, "Mekatronik");  // ❌ TEHLİKELİ! 5 < 11 → taşma!
```

5.5. strncpy — Güvenli String Kopyalama

Fonksiyon Tanımı:

```
char *strncpy(char *hedef, const char *kaynak, size_t n);
// hedef   : Hedef dizi
// kaynak  : Kaynak string
// n       : Kopyalanacak maksimum karakter sayısı
// Döndürür: hedef dizisinin adresini
```

Kullanım Örneği:

```
char kaynak[] = "Mekatronik";
char hedef[6];

strncpy(hedef, kaynak, 5);  // En fazla 5 karakter kopyala
hedef[5] = '\0';            // ⚠️ Manuel \0 ekleme ZORUNLU!
printf("%s\n", hedef);      // "Mekat"
```

⚠️ Dikkat: `strncpy`, n karakter kopyaladığında `\0` otomatik eklemez! Manuel olarak eklenmesi gerekir.

5.6. strcpy vs strncpy Karşılaştırması

Özellik	`strcpy`	`strncpy`
Boyut kontrolü	❌ Yapmaz	✅ n ile sınırlar
Buffer overflow riski	⚠️ Yüksek	✅ Düşük
\0 ekleme	✅ Otomatik	❌ Manuel gerekebilir
Performans	Hızlı	Biraz yavaş
Tercih	Güvenli ortam	Gömülü sistemler

```c
// Güvenli kopyalama kalıbı (pattern)
#define BUF_SIZE 20
char hedef[BUF_SIZE];
strncpy(hedef, kaynak, BUF_SIZE - 1);  // 1 byte \0 için ayır
hedef[BUF_SIZE - 1] = '\0';            // Garantili sonlandırma
```

5.7. strcat — String Birleştirme

Fonksiyon Tanımı:

```
1  char *strcat(char *hedef, const char *kaynak);
2  // hedef   : Sonuna eklenecek hedef string (yeterli boyutta olmalı!)
3  // kaynak  : Eklenecek kaynak string
4  // Döndürür: hedef dizisinin adresini
```

Kullanım Örneği:

```
1  char mesaj[50] = "Merhaba ";
2  strcat(mesaj, "Dünya!");
3  printf("%s\n", mesaj);  // "Merhaba Dünya!"
4
5  // Çoklu birleştirme
6  char yol[100] = "/home/";
7  strcat(yol, "kullanici/");
8  strcat(yol, "belgeler");
9  printf("%s\n", yol);  // "/home/kullanici/belgeler"
```

⚠️ Hedef dizinin, mevcut içerik + yeni içerik + `\0` için yeterli boyutta olması gerekir.

5.8. strncat — Güvenli String Birleştirme

Fonksiyon Tanımı:

```
1  char *strncat(char *hedef, const char *kaynak, size_t n);
2  // hedef   : Hedef string
3  // kaynak  : Eklenecek kaynak string
4  // n       : Eklenecek maksimum karakter sayısı
5  // Döndürür: hedef dizisinin adresini
```

Kullanım Örneği:

```
1  char buf[15] = "Hel";
2  strncat(buf, "lo World!!!", 5);  // En fazla 5 karakter ekle
3  printf("%s\n", buf);  // "Hello Wo" (3 + 5 = 8 karakter + \0)
```

✅ `strncat`, `strncpy`'den farklı olarak `\0`'ı otomatik olarak ekler.

5.9. strcmp — String Karşılaştırma

Fonksiyon Tanımı:

```c
int strcmp(const char *s1, const char *s2);
// s1, s2  : Karşılaştırılacak iki string
// Döndürür:
//   0      → s1 ve s2 EŞİT
//   < 0    → s1, s2'den KÜÇÜK (ASCII sırasında önce gelir)
//   > 0    → s1, s2'den BÜYÜK (ASCII sırasında sonra gelir)
```

Kullanım Örneği:

```c
printf("%d\n", strcmp("abc", "abc"));  //  0 (eşit)
printf("%d\n", strcmp("abc", "abd"));  // <0 ('c' < 'd')
printf("%d\n", strcmp("abd", "abc"));  // >0 ('d' > 'c')
printf("%d\n", strcmp("abc", "ABC"));  // >0 ('a'=97 > 'A'=65)
```

5.10. Neden `==` ile String Karşılaştırılmaz?

```c
char s1[] = "Merhaba";
char s2[] = "Merhaba";

// ❌ HATALI: == operatörü adresleri karşılaştırır, içeriği DEĞİL!
if (s1 == s2) {
    printf("Eşit\n");  // ÇALIŞMAZ! s1 ve s2 farklı adreslerdedir
}

// ✅ DOĞRU: strcmp ile karşılaştır
if (strcmp(s1, s2) == 0) {
    printf("Eşit\n");  // ← Doğru çalışır
}
```

İfade	Ne Karşılaştırır	Doğru Mu?
`s1 == s2`	Bellek adreslerini	❌
`strcmp(s1, s2) == 0`	Karakter içeriklerini	✅

5.11. strncmp — Kısmi String Karşılaştırma

Fonksiyon Tanımı:

```c
int strncmp(const char *s1, const char *s2, size_t n);
// İlk n karakteri karşılaştırır
```

Kullanım Örneği:

```c
char komut[] = "ATD123456";

// Sadece ilk 2 karakteri karşılaştır (AT komutu kontrolü)
if (strncmp(komut, "AT", 2) == 0) {
    printf("AT komutu algılandı: %s\n", komut);
}

// Dosya uzantısı kontrolü
char dosya[] = "veri.csv";
int uzunluk = strlen(dosya);
if (strncmp(dosya + uzunluk - 4, ".csv", 4) == 0) {
    printf("CSV dosyası tespit edildi!\n");
}
```

Mekatronik Kullanımı: UART üzerinden gelen AT komutlarının başlık kontrolü.

5.12. strchr ve strrchr — Karakter Arama

Fonksiyon Tanımları:

```c
char *strchr(const char *str, int c);   // İLK bulunan karakter
char *strrchr(const char *str, int c);  // SON bulunan karakter
// str : Aranacak string
// c   : Aranan karakter
// Döndürür: Bulunan karakterin adresi (bulunamazsa NULL)
```

Kullanım Örneği:

```c
char yol[] = "/home/kullanici/belgeler/rapor.txt";

char *son_slash = strrchr(yol, '/');  // Son '/' karakterini bul
if (son_slash != NULL) {
    printf("Dosya adı: %s\n", son_slash + 1);  // "rapor.txt"
}

char *nokta = strchr(yol, '.');  // İlk '.' karakterini bul
if (nokta != NULL) {
    printf("Uzantı: %s\n", nokta);  // ".txt"
}
```

5.13. strstr — Alt String Arama

Fonksiyon Tanımı:

```c
char *strstr(const char *samanlık, const char *igne);
// samanlık : İçinde arama yapılacak string
// igne     : Aranan alt string
// Döndürür : Alt stringin ilk bulunduğu adres (bulunamazsa NULL)
```

Kullanım Örneği:

```c
char gps[] = "$GPGGA,123519,4807.038,N,01131.000,E*47";

// GPS NMEA cümlesinde "GPGGA" var mı?
if (strstr(gps, "GPGGA") != NULL) {
    printf("GPS Fix verisi bulundu!\n");
}

// Hata mesajı kontrolü
char yanit[] = "ERROR: Sensor timeout at pin 7";
if (strstr(yanit, "ERROR") != NULL) {
    printf("Hata tespit edildi!\n");
}
```

5.14. memset — Bellek Bloğu Doldurma

Fonksiyon Tanımı:

```c
void *memset(void *ptr, int deger, size_t n);
// ptr   : Doldurulacak bellek bloğunun başlangıç adresi
// deger : Her byte'a yazılacak değer
// n     : Doldurulacak byte sayısı
// Döndürür: ptr adresini
```

Kullanım Örneği:

```c
// Diziyi sıfırlama
int dizi[100];
memset(dizi, 0, sizeof(dizi));  // Tüm byte'ları 0 yap
// Tampon temizleme
char tampon[256];
memset(tampon, 0, sizeof(tampon));  // Güvenli sıfırlama
// Belirli değerle doldurma (sadece char/byte dizileri için güvenli)
char yildizlar[20];
memset(yildizlar, '*', 19);
yildizlar[19] = '\0';
printf("%s\n", yildizlar);  // "******************"
```

5.15. memcpy — Bellek Bloğu Kopyalama

Fonksiyon Tanımı:

```
void *memcpy(void *hedef, const void *kaynak, size_t n);
// hedef  : Kopyalanacak hedef bellek alanı
// kaynak : Kopyalanacak kaynak bellek alanı
// n      : Kopyalanacak byte sayısı
// Döndürür: hedef adresini
```

Kullanım Örneği:

```
// Dizi kopyalama (strcpy'den farklı olarak her veri tipiyle çalışır)
int kaynak[] = {10, 20, 30, 40, 50};
int hedef[5];
memcpy(hedef, kaynak, sizeof(kaynak));  // 20 byte kopyala
// Struct kopyalama
typedef struct { int x, y; } Nokta;
Nokta p1 = {10, 20};
Nokta p2;
memcpy(&p2, &p1, sizeof(Nokta));
```

`memcpy`, `strcpy`'den daha genel amaçlıdır: string dışı verilerde (int, float, struct) de kullanılır.

5.16. string.h Fonksiyonları

Özet Tablosu

Fonksiyon	Açıklama	Prototip
`strlen(s)`	Uzunluk (\0 hariç)	`size_t strlen(const char *s)`
`strcpy(h, k)`	Kopyalama	`char *strcpy(char *h, const char *k)`
`strncpy(h, k, n)`	Güvenli kopyalama	`char *strncpy(char *h, const char *k, size_t n)`
`strcat(h, k)`	Birleştirme	`char *strcat(char *h, const char *k)`
`strncat(h, k, n)`	Güvenli birleştirme	`char *strncat(char *h, const char *k, size_t n)`
`strcmp(s1, s2)`	Karşılaştırma	`int strcmp(const char *s1, const char *s2)`
`strncmp(s1, s2, n)`	Kısmi karşılaştırma	`int strncmp(const char *s1, const char *s2, size_t n)`

5.16. string.h Fonksiyonları (devam)

Özet Tablosu

Fonksiyon	Açıklama	Prototip
`strchr(s, c)`	Karakter arama (ilk)	`char *strchr(const char *s, int c)`
`strrchr(s, c)`	Karakter arama (son)	`char *strrchr(const char *s, int c)`
`strstr(s, alt)`	Alt string arama	`char *strstr(const char *s, const char *alt)`
`memset(p, d, n)`	Bellek doldurma	`void *memset(void *p, int d, size_t n)`
`memcpy(h, k, n)`	Bellek kopyalama	`void *memcpy(void *h, const void *k, size_t n)`

5.17. Güvenli vs Güvensiz Fonksiyonlar

Güvensiz ⚠️	Güvenli ✅	Neden?
`strcpy(h, k)`	`strncpy(h, k, n)`	Boyut sınırı yok → buffer overflow
`strcat(h, k)`	`strncat(h, k, n)`	Boyut sınırı yok → buffer overflow
`sprintf(h, ...)`	`snprintf(h, n, ...)`	Boyut sınırı yok → buffer overflow
`gets(s)`	`fgets(s, n, stdin)`	Satır boyutu sınırı yok → ÇOK TEHLİKELİ

Kural: Gömülü sistemlerde ve güvenlik gerektiren yazılımlarda **her zaman sınırlı (n)** versiyonları tercih edin.

```
1  // ❌ gets() — C11'de kaldırıldı, ASLA kullanmayın!
2  // gets(tampon);  // Sınır yok → Saldırıya açık!
3
4  // ✅ fgets() — Güvenli alternatif
5  fgets(tampon, sizeof(tampon), stdin);
```

BÖLÜM 6

Uygulamalı Mekatronik Senaryoları

6.1. Hareketli Ortalama Filtresi — Görselleştirme

```
Pencere boyutu = 3 olduğunda:

Ham veri:      23.1  24.8  22.5  25.2  23.9  24.5  22.8  25.0  23.7  24.2
                     ├──────────────────┤
                            Pencere (i=2): (24.8+22.5+25.2)/3 = 24.17

Filtreli:      23.95 23.47 24.17 23.87 24.53 23.73 24.10 23.83 24.30 23.95
```

Sonuç: Anlık dalgalanmalar (gürültü) ortadan kalkar, sinyal **yumuşar**.

Mekatronik Uygulamaları:

- Sıcaklık sensörü okumalarının stabilizasyonu
- İvmeölçer verilerinin gürültü filtrelemesi
- Analog sensörlerde ADC okumalarını düzgünleştirme

6.2. Uygulama 1: Hareketli Ortalama Filtresi — Kod

Gürültülü sensör verisini yumuşatmak için en temel sinyal işleme tekniği:

```c
#include <stdio.h>
#define N 10        // Veri sayısı
#define PENCERE 3   // Filtre pencere boyutu

int main() {
    double sensor[N] = {23.1, 24.8, 22.5, 25.2, 23.9,
                        24.5, 22.8, 25.0, 23.7, 24.2};
    double filtreli[N];

    int yari = PENCERE / 2;
    for (int i = 0; i < N; i++) {
        double toplam = 0;
        int sayac = 0;
        for (int j = -yari; j <= yari; j++) {
            if (i + j >= 0 && i + j < N) {
                toplam += sensor[i + j];
                sayac++;
            }
        }
        filtreli[i] = toplam / sayac;
```

6.3. Uygulama 2: Dizide Değer Sayma ve Eşik Kontrolü

Bir sensör dizisinde belirli bir eşik değerini aşan okumaları sayma:

```c
#include <stdio.h>
#define N 10

int main() {
    int sicaklik[N] = {22, 25, 31, 28, 35, 24, 33, 29, 36, 27};
    int esik = 30;
    int sayac = 0;

    printf("Eşik değeri: %d°C\n", esik);
    printf("Eşiği aşan okumalar: ");

    for (int i = 0; i < N; i++) {
        if (sicaklik[i] > esik) {
            printf("%d°C ", sicaklik[i]);
            sayac++;
        }
    }

    printf("\nToplam %d okuma eşiği aştı.\n", sayac);
    printf("Alarm oranı: %%%.0f\n", (double)sayac / N * 100);
```

Çıktı:

```
Çıktı:
Eşik değeri: 30°C
Eşiği aşan okumalar: 31°C 35°C 33°C 36°C
Toplam 4 okuma eşiği aştı.
Alarm oranı: %40
```

6.4. Uygulama 3: Bubble Sort (Kabarcık Sıralama)

Sensör verilerini sıralamak — en basit sıralama algoritması:

```c
#include <stdio.h>
#define N 8

int main() {
    int dizi[N] = {64, 34, 25, 12, 22, 11, 90, 45};

    printf("Sıralama öncesi: ");
    for (int i = 0; i < N; i++) printf("%d ", dizi[i]);
    printf("\n");

    // Bubble Sort
    for (int i = 0; i < N - 1; i++) {
        for (int j = 0; j < N - 1 - i; j++) {
            if (dizi[j] > dizi[j + 1]) {
                // Swap (takas)
                int gecici = dizi[j];
                dizi[j] = dizi[j + 1];
                dizi[j + 1] = gecici;
            }
        }
```

6.5. Bubble Sort — Adım Adım İzleme

`dizi = {64, 34, 25, 12}` için sıralama adımları:

```
Tur 1 (i=0): En büyük eleman sona kayar
  [64, 34, 25, 12] → 64>34? Evet → [34, 64, 25, 12]
  [34, 64, 25, 12] → 64>25? Evet → [34, 25, 64, 12]
  [34, 25, 64, 12] → 64>12? Evet → [34, 25, 12, 64] ← 64 yerleşti

Tur 2 (i=1): Sondan bir önceki yerleşir
  [34, 25, 12, 64] → 34>25? Evet → [25, 34, 12, 64]
  [25, 34, 12, 64] → 34>12? Evet → [25, 12, 34, 64] ← 34 yerleşti

Tur 3 (i=2): Tüm dizi sıralı
  [25, 12, 34, 64] → 25>12? Evet → [12, 25, 34, 64] ✅ Tamamlandı!
```

Karmaşıklık: $O(n^2)$ — Büyük dizilerde yavaştır, ancak öğretim amaçlı idealdir.

6.6. Uygulama 4: Selection Sort (Seçerek Sıralama)

```c
#include <stdio.h>
#define N 6

int main() {
    int dizi[N] = {29, 10, 14, 37, 13, 25};

    for (int i = 0; i < N - 1; i++) {
        int minIdx = i;  // Minimum elemanın indeksini bul
        for (int j = i + 1; j < N; j++) {
            if (dizi[j] < dizi[minIdx]) {
                minIdx = j;
            }
        }
        // Minimum elemanı i. pozisyona taşı (swap)
        if (minIdx != i) {
            int gecici = dizi[i];
            dizi[i] = dizi[minIdx];
            dizi[minIdx] = gecici;
        }
    }
```

Farkı: Her turda **minimum elemanı bulur** ve yerine koyar. Takas sayısı O(n).

6.7. Uygulama 5: Sensör Verisi İstatistikleri

```c
#include <stdio.h>
#include <math.h>
#define N 10

int main() {
    double sicaklik[N] = {22.5, 23.1, 22.8, 24.0, 23.5,
                          22.9, 23.7, 24.2, 22.6, 23.3};

    // 1. Ortalama
    double toplam = 0;
    for (int i = 0; i < N; i++) toplam += sicaklik[i];
    double ort = toplam / N;

    // 2. Standart Sapma
    double varyans = 0;
    for (int i = 0; i < N; i++) {
        varyans += (sicaklik[i] - ort) * (sicaklik[i] - ort);
    }
    varyans /= N;
    double stdSapma = sqrt(varyans);
```

6.8. Uygulama 6: Median (Ortanca) Hesaplama

Medyan, verileri sıralayıp ortadaki değeri almaktır. Aykırı değerlere (outlier) karşı ortalamadan daha dayanıklıdır:

```c
#include <stdio.h>
#define N 7

int main() {
    int veri[N] = {45, 12, 78, 23, 56, 34, 67};

    // 1. Diziyi sırala (Bubble Sort)
    for (int i = 0; i < N - 1; i++) {
        for (int j = 0; j < N - 1 - i; j++) {
            if (veri[j] > veri[j + 1]) {
                int tmp = veri[j];
                veri[j] = veri[j + 1];
                veri[j + 1] = tmp;
            }
        }
    }
    // Sıralı: {12, 23, 34, 45, 56, 67, 78}

    // 2. Medyanı bul
    double medyan;
```

6.9. Uygulama 7: Matris Çarpımı — Kinematik Dönüşüm

Robotik kolda iki eklem dönüşüm matrisinin çarpılması:

```c
#include <stdio.h>
#define N 3

int main() {
    // 30° döndürme matrisi (cos30=0.866, sin30=0.5)
    double R1[N][N] = {
        { 0.866, -0.500, 0.0},
        { 0.500,  0.866, 0.0},
        { 0.0,    0.0,   1.0}
    };
    // 45° döndürme matrisi
    double R2[N][N] = {
        { 0.707, -0.707, 0.0},
        { 0.707,  0.707, 0.0},
        { 0.0,    0.0,   1.0}
    };
    double sonuc[N][N] = {0};

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
```

6.10. Uygulama 8: Frekans Analizi (Histogram)

Bir dizideki değerlerin frekans dağılımını hesaplama:

```c
#include <stdio.h>
#define N 20
#define MAX_DEGER 10

int main() {
    int veri[N] = {3, 7, 2, 5, 3, 7, 3, 8, 5, 2,
                   7, 3, 5, 8, 2, 3, 7, 5, 8, 3};

    int frekans[MAX_DEGER + 1] = {0};  // Sayaç dizisi

    // Frekans hesapla
    for (int i = 0; i < N; i++) {
        frekans[veri[i]]++;
    }

    // Histogram çiz
    printf("Değer | Frekans | Grafik\n");
    printf("------+---------+----------\n");
    for (int i = 0; i <= MAX_DEGER; i++) {
        if (frekans[i] > 0) {
```

Dizi Seçim Rehberi — Özet Tablosu

Soru	Cevap	Dizi Türü
Aynı tipte çok veri mi saklayacaksınız?	Evet	1D dizi
Tablo/matris yapısı mı gerekli?	Evet	2D dizi
Metin/komut işleyecek misiniz?	Evet	char dizisi (string)
Boyut derleme zamanında belli mi?	Evet	Sabit boyutlu dizi
Boyut çalışma zamanında belirleniyor mu?	Evet	VLA veya malloc (ileri konu)

Sık Yapılan Dizi Hataları

1. **Sınır aşımı:** `dizi[N]` erişimi → N eleman varsa son indeks `N-1`
2. **Başlatma unutma:** Yerel diziler çöp değer içerir → `{0}` ile başlatın
3. **sizeof yanılgısı:** `sizeof(dizi)` toplam byte verir, eleman sayısı **değildir** → `sizeof(dizi)/sizeof(dizi[0])` kullanın
4. **String \0 eksikliği:** Null-terminator olmadan `printf` çöp veri yazdırır
5. **== ile string karşılaştırma:** `strcmp` kullanılmalıdır
6. **Kopyalama hatası:** `dizi2 = dizi1` yazılamaz → döngü veya `memcpy`

```c
// HATA: Off-by-one — Son eleman sınır dışı!
int dizi[5] = {1, 2, 3, 4, 5};
for (int i = 0; i <= 5; i++) {  // ← i=5 sınır dışı!
    printf("%d ", dizi[i]);
}
// ✅ DOĞRU: i < 5 veya i <= 4
```

Teşekkürler

Soru ve Cevaplar

Marmara Üniversitesi – Bilgisayar Programlama Dersi Hafta 6: Diziler (Tek Boyutlu, Çok Boyutlu, Karakter Dizileri ve Mekatronik Uygulamaları)